

TECHNICAL DOCUMENTATION

# Theory, design & architecture of **CommitX.**

A complete reference to the protocol, smart contract, backend, frontend, security model, testing and deployment — with answers to the questions asked in technical interviews.

**14 sections**  
End to end

**Solidity + Next.js**  
Full stack

**Sepolia**  
Chain 11155111

**October 2026**  
Edition

# What's inside

|    |                                 |
|----|---------------------------------|
| 01 | Executive summary               |
| 02 | Problem & theory                |
| 03 | System architecture             |
| 04 | Smart contract: CommitXProtocol |
| 05 | Backend                         |
| 06 | Moderation & reputation         |
| 07 | Frontend                        |
| 08 | Key flows                       |
| 09 | Security & threat model         |
| 10 | Testing & quality               |
| 11 | Deployment                      |
| 12 | Design decisions & trade-offs   |
| 13 | Interview questions & answers   |
| 14 | Appendix: repository map        |

**CommitX** is a non-custodial accountability platform on Ethereum (Sepolia testnet). People commit to a goal, stake ETH in a smart contract, prove progress every period, get verified by the other participants, and are paid out by a deterministic on-chain settlement: you keep the share of your stake you earned, and people who finish enough of the challenge split what the others forfeited.

## What makes it interesting

- Hybrid design: verification is off-chain (cheap, flexible), money is on-chain (trust-minimised).
- EIP-712 signed settlement — the backend proposes results, the contract verifies the signature and does all money math.
- Exact integer-wei accounting with a provable conservation invariant.
- Peer review with moderation: complaints, reputation, suspension and removal.

## Stack

|          |                                                                                 |
|----------|---------------------------------------------------------------------------------|
| Contract | Solidity 0.8.24 (vialR, cancun), OpenZeppelin v5, Hardhat                       |
| App      | Next.js 14 App Router, React 18, Tailwind CSS 3                                 |
| Web3     | ethers v6 (only where transactions happen), raw EIP-1193 elsewhere              |
| Data     | Prisma 5 · SQLite locally · PostgreSQL in production                            |
| Auth     | Wallet signature sessions (EIP-191 + HMAC cookie); optional Google via Supabase |
| Deploy   | Vercel (serverless) or Docker (standalone server)                               |

## Live deployment (Sepolia, chain 11155111)

| CONTRACT     | ADDRESS                                    | CHALLENGE IDS | FEATURES                                            |
|--------------|--------------------------------------------|---------------|-----------------------------------------------------|
| Current (v3) | 0x40aEd3065673975ad7a52C71ACa1FCBBb41bE681 | 2001+         | 0.125% fee, under-filled refund, admin account lock |
| Previous     | 0x92260222110C5EB79a5Cd2989c0D03CcC92658FC | 1001–2000     | 0.125% fee                                          |

## The problem

People routinely fail at goals they genuinely want (exercise, study, shipping code). Behavioural economics explains why:

- **Present bias / hyperbolic discounting** — the cost of effort is now, the benefit is later, so "tomorrow" always wins.
- **Loss aversion** — losses weigh roughly twice as much as equal gains (Kahneman & Tversky). Losing a stake motivates more than an equal reward.
- **Commitment devices** — voluntarily restricting your future options (StickK, Beeminder) measurably improves follow-through.
- **Social accountability** — being observed by peers increases compliance.

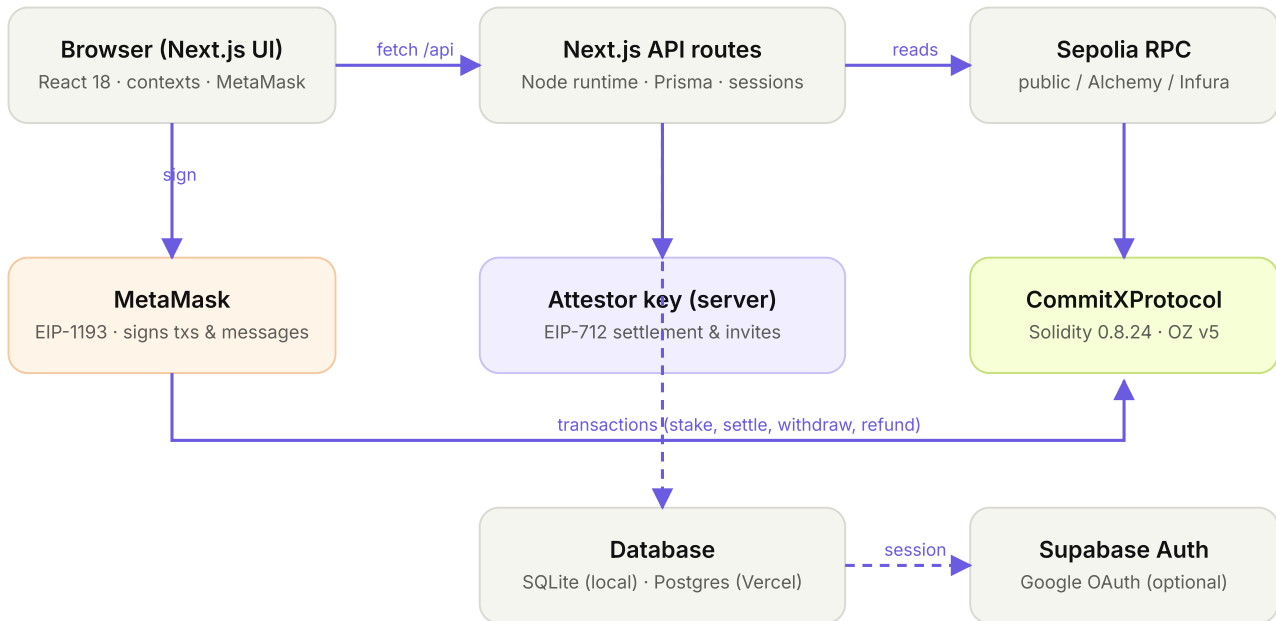
CommitX combines all four: a **stake** (loss aversion + commitment), a fixed **schedule of periods** (small, frequent wins), and **peer verification** (social accountability).

## Why a blockchain?

A commitment device only works if the money is credibly at risk and the payout rules can't be changed afterwards. A smart contract gives: custody nobody can run away with, rules published in code, and payouts anyone can audit. Traditional platforms require trusting the company with both your money and the judging.

## Mechanism design

- **Proportional retention:** each verified period keeps  $1/T$  of your stake — progress is always rewarded, so dropping out early never becomes "rational".
- **Penalty pool:** missed periods fund a pool split among **qualifiers** ( $\geq$  threshold %), weighted by periods completed — finishing is strictly better than partially finishing.
- **Zero-qualifier rule:** if nobody qualifies, the pool goes to the treasury. This prevents collusion like "everyone skips and we share the pool".
- **Peer review incentives:** reviews earn reputation (+2); wrong reviews and false complaints cost reputation (-10) and can cost the stake (removal).
- **Minimum two people:** a lone participant can't be verified, so an under-filled challenge returns the stake (minus the fee) instead of running.



Solid arrows: runtime calls. Dashed: data/auth relationships.

## Hybrid on-chain / off-chain split

### ON-CHAIN (CONTRACT)

Custody of every wei, joins, fees, settlement math, withdrawals, refunds, attestor signature check, replay protection

### OFF-CHAIN (NEXT.JS + DB)

Challenge metadata, period schedule, proofs, votes, complaints, reputation, profiles, invitations, notifications

**Why:** storing proofs and votes on-chain would cost gas for every action and expose personal links publicly. The expensive, trust-critical part (money) stays on-chain; the cheap, flexible part (judgement) stays off-chain and is **summarised** into one signed message at settlement.

## Trust model

| COMPONENT              | CAN                                                                                  | CANNOT                                                                                |
|------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Backend / attestor     | Sign which participants completed how many periods                                   | Move funds, change stakes or rules, settle twice, sign for a different contract/chain |
| Admin (contract owner) | Rotate attestor/treasury, lock accounts, claim treasury balance, moderate complaints | Withdraw participants' stakes or payouts                                              |
| Participants           | Stake, prove, review, report, trigger settlement, withdraw their own funds           | Vote on their own proof, vote twice, act as another wallet                            |

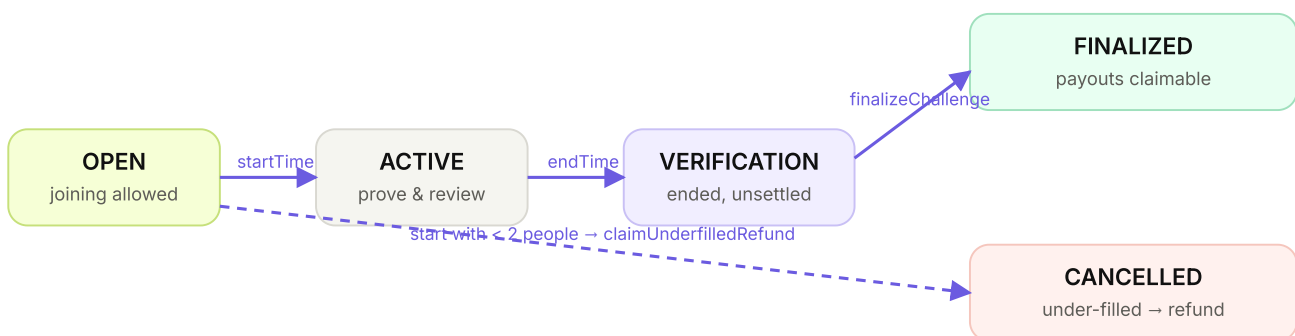
The residual trust is the attestor: a compromised key could sign wrong period counts (though never move funds directly). Mitigations and alternatives are discussed in *Security*.

## State

```
struct Challenge {
    uint256 id; address creator; uint256 stakeAmount;    // stake per participant (wei)
    uint16 totalPeriods; uint8 qualificationThreshold;    // e.g. 30 periods, 50%
    uint32 startTime; uint32 endTime; uint32 verificationDuration;
    uint16 maxParticipants; uint16 participantCount;
    uint256 settlementNonce;                               // EIP-712 replay protection
    ChallengeStatus status; bool finalized; bool isPrivate; string metadataURI;
}
mapping(uint256 ⇒ Challenge) challenges;
mapping(uint256 ⇒ mapping(address ⇒ bool)) isParticipant, hasWithdrawn, isWhitelisted;
mapping(uint256 ⇒ mapping(address ⇒ uint256)) claimable;    // pull-payment ledger
mapping(address ⇒ bool) isLocked;                          // admin lock (v3)
uint256 treasuryBalance, totalFeesCollected, totalDeposited, totalSettledAmount,
totalWithdrawnAmount;
```

Struct fields are packed (uint16/uint32/uint8) so several share one storage slot — fewer SSTORES on creation.

## Lifecycle



`getCurrentStatus()` derives OPEN/ACTIVE/VERIFICATION from timestamps, so no transaction is needed to "start" a challenge.

## External functions

| FUNCTION                                             | WHO     | PURPOSE / KEY CHECKS                                                                                          |
|------------------------------------------------------|---------|---------------------------------------------------------------------------------------------------------------|
| <code>createChallenge(...)</code>                    | anyone  | start in future, end > start, threshold 1–100, max ≥ 2; optional auto-join if <code>msg.value == stake</code> |
| <code>joinChallenge(id)</code>                       | anyone  | OPEN, before start, not full, not joined, exact stake, whitelist if private                                   |
| <code>joinChallengeWithAuthorization(id, sig)</code> | invitee | private join with an EIP-712 <code>Invitation(challengeId, participant)</code> signed by creator or attestor  |

|                                                                                           |             |                                                                                                                                      |
|-------------------------------------------------------------------------------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <code>finalizeChallenge(id, nonce, addrs[], periods[], sig)</code>                        | anyone      | after end, $\geq 2$ participants, strictly ascending enrolled addresses, periods $\leq T$ , valid attestor signature, matching nonce |
| <code>withdraw(id)</code>                                                                 | participant | pull payment; not locked; zero-before-transfer (CEI) + nonReentrant                                                                  |
| <code>claimUnderfilledRefund(id)</code>                                                   | participant | after start with $< 2$ participants: returns stake – fee; marks CANCELLED                                                            |
| <code>withdrawTreasury(amount)</code>                                                     | treasury    | fees + zero-qualifier pools + rounding dust                                                                                          |
| <code>setAccountLock</code> / <code>setProtocolAttestor</code> / <code>setTreasury</code> | owner       | administration                                                                                                                       |
| <code>whitelistParticipants</code> / <code>revokeWhitelist</code>                         | creator     | private access before start                                                                                                          |

## Settlement math (exact integer wei)

```

fee      = stake × 125 / 100000      (0.125%, taken at join → treasury)
S        = stake - fee               (net stake used for settlement)
retained_i = S × c_i / T              (c_i = verified periods of i)
pool      = Σ (S - retained_i)
qualifier ⇔ c_i × 100 ≥ threshold × T (integer comparison, no division)
reward_i  = pool × c_i / Σ c_q       (qualifiers only)
claimable_i = retained_i + reward_i
treasury += pool - Σ reward_i        (zero-qualifier pool, or rounding dust)

```

**Worked example.** Stake 0.1 ETH → fee 0.000125, S = 0.099875. T = 10, threshold 50%. Completed: A 10, B 9, C 7, D 2. Retained: 0.099875, 0.0898875, 0.0699125, 0.019975. Pool = 0.1198500. Qualifiers A, B, C (weights 26). Rewards: A 0.0460962, B 0.0414865, C 0.0322673; D 0. Sum of claimables + dust = 4·S exactly; plus 4 fees = 4 stakes deposited.

## Conservation invariant

For every finalized challenge:  $\Sigma \text{claimable} + \text{treasuryCut} + \Sigma \text{fees} = n \times \text{stake}$ . Floor division always rounds *down*, and the remainder is assigned to the treasury, so the contract can never owe more than it holds. The test suite asserts this to the wei.

## EIP-712 typed data

```

Domain:      { name: "CommitX", version: "1", chainId: 11155111, verifyingContract: <contract> }
Settlement:  (uint256 challengeId, uint256 settlementNonce, address[] participants, uint256[]
completedPeriods)
Invitation:  (uint256 challengeId, address participant)

```

Arrays are hashed as `keccak256(abi.encodePacked(elements as 32-byte words))`, matching the EIP-712 spec for dynamic arrays. Binding `chainId` and `verifyingContract` prevents cross-chain and cross-contract replay; the per-challenge `settlementNonce` (incremented on `finalize`) plus `finalized` prevents re-use.

## Canonical ordering

The contract requires participant addresses in strictly ascending order. One check (`p > last`) proves there are no duplicates, and the signer and verifier agree on a single canonical array, so the signature is deterministic.

## Security patterns

- **Pull payments:** settlement only writes balances; each user withdraws. One failing recipient can't block others; no loops send ETH.
- **Checks-Effects-Interactions + ReentrancyGuard** on every function that sends ETH.
- **Custom roles:** Ownable (admin), attester, treasury, creator — least privilege.
- **Exact-value joins** (`msg.value == stake`) — no change to refund, no accounting drift.

## Versions

v1 (no fee) → v2 (0.125% fee, under-filled refund, IDs from 1001) → v3 (admin account lock, IDs from 2001).  
Deployed contracts are immutable, so new versions are deployed alongside; `src/config/deployments.js` maps ID ranges to contracts and the app routes each challenge to its own contract. The deploy script picks a non-colliding first ID automatically and keeps older contracts routable.

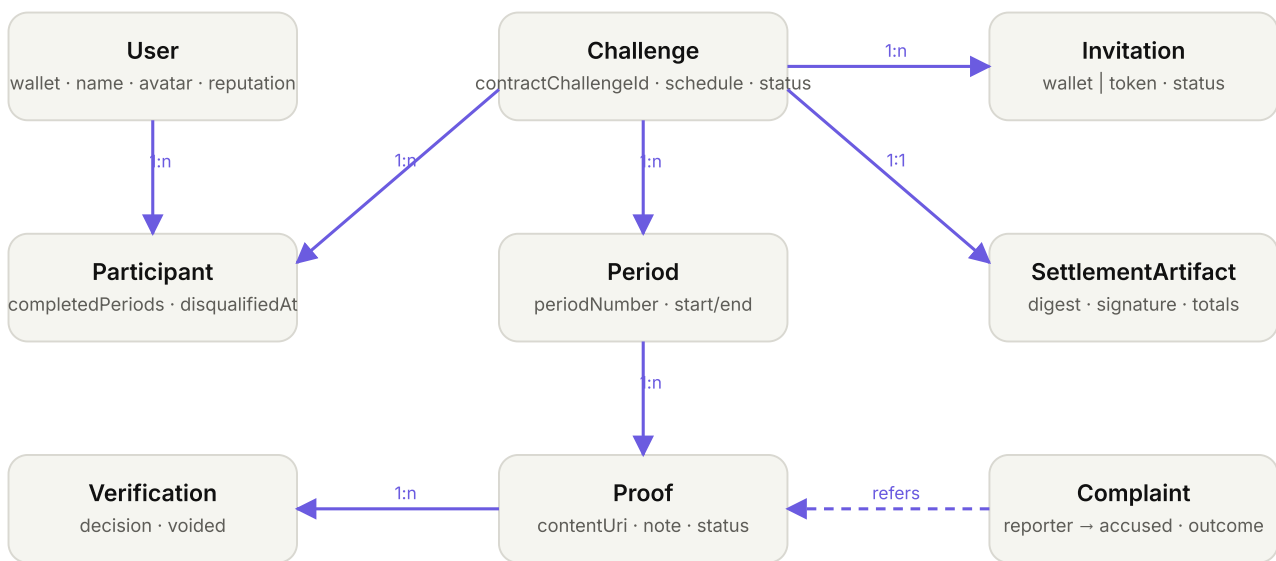


The backend is the set of Next.js App Router route handlers under `src/app/api`, running on Node with Prisma. It never holds funds.

## API surface

| ROUTE                                                                          | METHOD          | AUTH                       | PURPOSE                                                              |
|--------------------------------------------------------------------------------|-----------------|----------------------------|----------------------------------------------------------------------|
| <code>/api/auth/nonce</code> · <code>/verify</code> · <code>/session</code>    | GET/POST/DELETE | —                          | Wallet sign-in (one-time message → HMAC session cookie)              |
| <code>/api/challenges</code>                                                   | GET/POST        | — (POST verified on-chain) | List (with chain sync) · register metadata for an on-chain challenge |
| <code>/api/challenges/:id</code>                                               | GET             | —                          | Detail; self-heals from chain if missing; redacts invite tokens      |
| <code>/api/challenges/:id/join</code>                                          | POST            | verified on-chain          | Mirror an enrolment that exists on-chain                             |
| <code>/api/challenges/:id/authorize-join</code>                                | POST            | invite + reputation        | EIP-712 invitation signature for private joins                       |
| <code>/api/challenges/:id/invitations[:inviteId]</code>                        | GET/POST/DELETE | creator session            | Manage invites (before start only)                                   |
| <code>/api/proofs</code>                                                       | GET/POST        | wallet session             | Submit proof (open period, one per period) · list                    |
| <code>/api/verifications</code>                                                | POST            | wallet session             | Approve/reject a peer's proof                                        |
| <code>/api/attestation/:id</code>                                              | POST            | —                          | Build & sign the settlement                                          |
| <code>/api/complaints</code> · <code>/api/reputation</code>                    | POST · GET      | session · —                | Report a vote/person · reputation lookup                             |
| <code>/api/admin/complaints[:id]</code> · <code>/participants</code>           | GET/POST        | admin session              | Moderation                                                           |
| <code>/api/profile</code> · <code>/api/stats</code> · <code>/api/health</code> | GET/POST        | session for POST           | Profile · protocol stats · liveness                                  |

## Data model



Wallet addresses are stored lowercase and used as natural keys between tables. Indexes cover the hot lookups (proofs by period+participant, participants by wallet, complaints by accused/status, reputation events by wallet+time).

## Authentication: wallet sessions

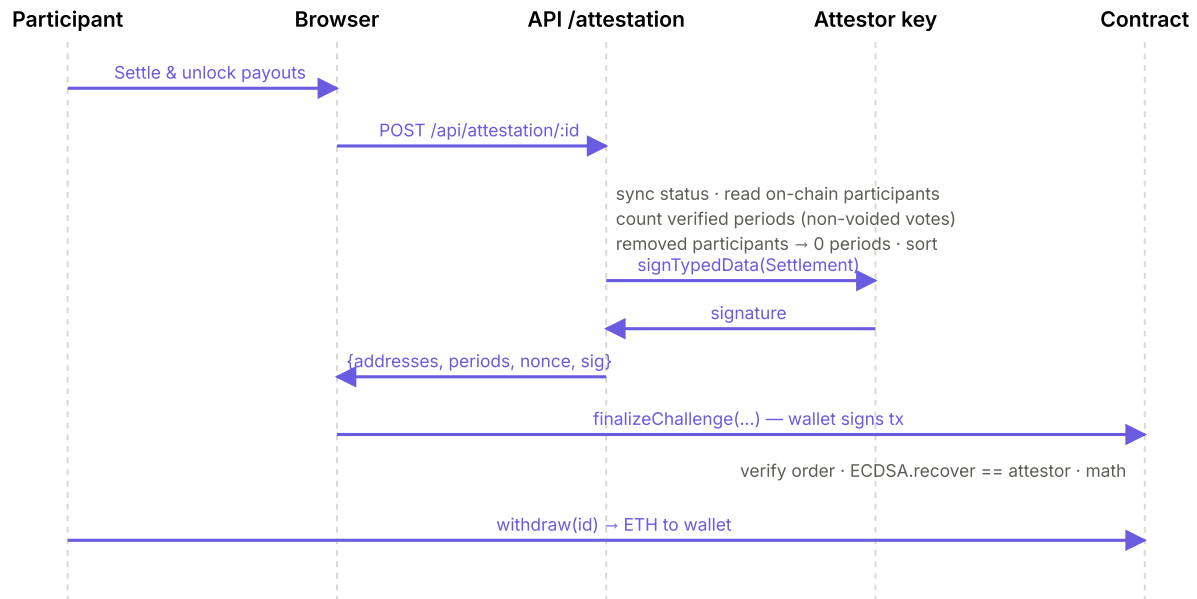
1. GET /api/auth/nonce stores a human-readable message with a random nonce and timestamp on the user row.
2. The browser calls `personal_sign` (EIP-191) through MetaMask — free, no transaction.
3. POST /api/auth/verify recovers the signer with `ethers.verifyMessage`, checks it matches, enforces a 10-minute TTL, clears the nonce (single use) and sets `cx_wallet = wallet.exp.HMAC_SHA256(secret)` — httpOnly, SameSite=Lax, Secure in production, 7 days.
4. Write endpoints call `requireWallet(request, claimedWallet)`; a mismatch returns 401 `WALLET_AUTH`, which the client's `apiFetch` answers by signing in and retrying once.

Timing-safe comparison is used for the HMAC. The server never trusts an address from the request body.

## Chain sync (the contract is the source of truth)

- Reads `getChallenge` + `getCurrentStatus` through a JSON-RPC provider, routed to the challenge's own contract.
- 15-second in-memory cache, 4-second timeout per RPC call (a slow node can't hang a page), settled/cancelled challenges skip RPC entirely.
- If a challenge exists on-chain but not in the DB (tab closed mid-create), it is rebuilt from on-chain data on first view (*self-heal*).
- Economic fields (creator, stake, schedule, capacity, privacy) are always taken from the contract, never from the request.

## Attestation pipeline



1. Re-sync status and nonce from chain; refuse if settled, cancelled or before end.
2. Read the participant list **from the contract** (the DB may have drifted) and backfill missing rows.
3. For every proof, count non-voided votes from enrolled, non-self, de-duplicated verifiers; approved if  $\geq \lfloor 0.33n \rfloor$  (min 1) and approvals > rejections.
4. Count **distinct periods** per participant (multiple proofs in one period count once); removed participants get 0.
5. Sort canonically, compute the accounting preview, sign Settlement, persist a SettlementArtifact (digest, signature, totals) for audit.

## Proof & review rules

- Proof: wallet session, participant, not removed, not suspended, challenge not settled, period currently open (+5 min grace), one per period, https link required.
- Review: wallet session, participant of the same challenge, not the proof owner, not removed, not suspended, one vote per proof (DB unique constraint).
- Each vote re-scores the proof and updates the owner's completedPeriods.

## Complaints

Any participant can report a specific vote (approved invalid proof / rejected valid proof) or a participant. Reports need a 10–1000 character explanation; duplicates and more than 10 open reports per person are refused. Only the **admin** (the contract owner(), read from chain) can resolve them.

| ADMIN ACTION    | EFFECT                                               |
|-----------------|------------------------------------------------------|
| Uphold          | Vote voided → proof re-scored → voter -10 reputation |
| Uphold & remove | Same + voter removed from the challenge              |
| False report    | Reporter -10 reputation                              |
| False & remove  | Same + reporter removed from the challenge           |
| Dismiss         | No penalty                                           |

## Removal (forfeit without refund)

A removed participant can no longer submit proof, review or report in that challenge. At settlement the attestor assigns them **0 completed periods**, so the contract computes `retained = 0` and no reward: their whole net stake flows into the penalty pool for honest qualifiers (or the treasury). No contract change was needed — the penalty is expressed through the signed settlement the contract already trusts. Removal is refused once a challenge is settled (payouts are final), and can be undone (reinstate) before settlement.

## Reputation

```

start      100   (range 0-100, stored on User, every change logged in ReputationEvent)
+2         each review cast
-10        upheld complaint against your vote, or your complaint ruled false
< 25       SUSPENDED: no joining, creating, submitting, reviewing or reporting
recovery   every full 4 days below 25 → +2 automatically, until back at 25
levels     ≥80 Trusted · ≥50 Fair · ≥25 At risk · <25 Suspended

```

Recovery is computed lazily: on any read, the server applies the +2 ticks owed since `lowSince` and logs them — no cron job needed (important on serverless). Enforcement is server-side on every write endpoint and in the UI; joining/creating are on-chain actions, so the app blocks them in the UI and refuses private-join signatures, while a suspended user who bypasses the UI still can't submit or review (stake would be lost).

## Structure

| AREA               | FILES                                                                                                                                                                                                                                                                                    |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pages (App Router) | <code>/</code> , <code>/explore</code> , <code>/create</code> , <code>/challenges/[id]</code> , <code>/dashboard</code> , <code>/verify</code> , <code>/profile</code> , <code>/learn</code> , <code>/protocol</code> , <code>/admin</code>                                              |
| Contexts           | <code>ToastContext</code> , <code>Web3Context</code> , <code>AuthContext</code> , <code>InvitationsContext</code> , <code>ProfileContext</code>                                                                                                                                          |
| UI kit             | <code>primitives</code> ( <code>Stat</code> , <code>Segmented</code> , <code>CountUp</code> , <code>ProgressRing</code> , <code>InfoTip...</code> ), <code>Avatar</code> , <code>DateTimePicker</code> , <code>Badge</code> , <code>Reveal</code>                                        |
| Libraries          | <code>lib/chain</code> ( <code>ethers</code> , <code>writes</code> ), <code>lib/network</code> ( <code>addresses</code> , <code>fee math</code> ), <code>lib/api</code> ( <code>apiFetch</code> + <code>wallet sign-in</code> ), <code>lib/formatters</code> , <code>lib/identity</code> |

## Web3 integration

- **Reads** go through a public RPC (`getReadContract(id)`), independent of the wallet's network — balances never read as zero because MetaMask is on the wrong chain.
- **Writes** (`getWriteContract(id)`) switch MetaMask to Sepolia first (adding it if missing), so one click completes an action.
- Withdrawals dry-run with `staticCall` first so a revert shows a human reason instead of a gas-estimation error.
- `friendlyTxError` maps revert strings and wallet codes (4001, -32002...) to plain sentences.

## Performance

| TECHNIQUE                                                                                                                     | RESULT                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Removed ethers from the global bundle (EIP-1193 calls + <code>BigInt</code> formatter); contract ABI moved out of shared code | Explore/Verify/Profile/Learn first-load JS $\approx$ 440 KB $\rightarrow$ $\approx$ 112 KB |
| Supabase SDK loaded lazily after first paint                                                                                  | Faster interactivity on every page                                                         |
| Middleware refreshes Supabase sessions only when an <code>sb-</code> cookie exists, and skips data APIs                       | No network round-trip per request                                                          |
| Chain cache + timeouts, skip terminal challenges, DB indexes                                                                  | Warm API responses $\approx$ 5–10 ms                                                       |
| Static pages prerendered, fonts self-hosted by next/font                                                                      | $\approx$ 30 ms page responses in production                                               |

## UX & design system

- Design tokens in Tailwind; the accent colour is a CSS variable (`--accent`) chosen per user and applied by an inline script before first paint.
- Dashboard "What to do next" derives tasks (submit, review, settle, withdraw, refund) from challenges + on-chain positions.
- Generated avatars: a seeded PRNG (FNV-1a + mulberry32) draws a deterministic "blob buddy" SVG per wallet; friendly default names ("Swift Otter").

- Custom date-time picker, journey stepper, tooltips, toasts, confetti — all respecting `prefers-reduced-motion`.
- Mobile bottom tab bar; no horizontal overflow at 375 px; error, 404 and loading boundaries.

## Create

1. Wizard (basics → schedule → stakes → people → review) computes  $T = \lfloor (\text{end} - \text{start}) / \text{frequency} \rfloor$ .
2. `createChallenge` tx (auto-stake optional) → ID parsed from the `ChallengeCreated` event.
3. POST metadata (retrieved while the public RPC catches up); the server reads all economic fields from the contract.

## Join

1. Public: `joinChallenge` with exact stake. Private: server checks invite + reputation, returns an EIP-712 invitation, wallet calls `joinChallengeWithAuthorization`.
2. App mirrors the enrolment only after confirming `isParticipant` on-chain.

## Prove & review

1. During each period: submit a link + note. Peers approve/reject (one vote each, never on your own proof).
2.  $\geq \lfloor 0.33n \rfloor$  approvals (min 1) and more approvals than rejections → period verified.

## Settle & withdraw

1. After the end anyone clicks "Settle": attestation is generated and `finalizeChallenge` is sent.
2. Each participant withdraws their claimable balance (pull payment).

## Under-filled

If fewer than two people joined by the start, the lone participant calls `claimUnderfilledRefund` and receives the stake minus the fee.

| THREAT                                               | MITIGATION                                                                                   |
|------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Reentrancy on withdraw/refund                        | CEI ordering + OpenZeppelin ReentrancyGuard                                                  |
| Settlement replay (same/different contract or chain) | EIP-712 domain (chainId, verifyingContract), per-challenge nonce, finalized flag             |
| Forged/duplicated participant list                   | Strictly ascending addresses, enrolment check, count must equal participantCount             |
| Impersonation in API calls                           | Signed wallet sessions; addresses from the body are only accepted if they match the session  |
| Fake joins / fake challenges in the DB               | Server verifies on-chain before recording; economic fields read from chain                   |
| Signature replay for sign-in                         | Single-use nonce with 10-minute TTL                                                          |
| Leaking private invite links / emails                | Tokens redacted unless the viewer is the creator; public endpoints select only name + avatar |
| Inflated payouts via many proofs                     | One proof per period; distinct-period counting at settlement                                 |
| Self-voting, double voting                           | Server checks + DB unique (proofId, verifierAddress); attestation re-audits                  |
| Bad validators / false reporters                     | Complaints, admin rulings, -10 reputation, suspension, removal with forfeiture               |
| Missing secret falls back to a public key            | Attestor key required off local chains; SESSION_SECRET required in production                |
| Open redirect after OAuth                            | Only same-site relative next paths                                                           |

## Known limitations (honest answers)

- **Attestor trust:** a compromised attestor key could sign wrong period counts. Improvements: multisig/threshold attestors, a challenge window where participants can dispute an attestation on-chain, or on-chain voting for small cohorts.
- **Collusion / Sybil:** one person with several wallets could approve their own proofs. Mitigations: reputation, private cohorts, proof-of-personhood, stake-weighted or randomised reviewers.
- **Proof authenticity:** links are judged by people; integrations (GitHub/Strava APIs, oracles) could automate verification.
- **Reputation is off-chain,** so joining/creating can only be blocked in the app, not by the contract.



| SUITE                       | COVERAGE                                                                                                                                                                                                                                                                                                                                                        |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hardhat contract tests (41) | Roles; creation validation; joins (stake, duplicate, full, after start); under-filled refunds; EIP-712 ordering, duplicates, wrong signer, replay, before-end; wei conservation with qualifiers and zero-qualifier; treasury; private challenges & invitation signatures; fixed start; 15-period settlement; fee math & admin claims; ID offsets; account locks |
| API security suite (16)     | Impersonation, fake joins, fake challenges, unsigned wallet linking, settle-twice, replayed sign-in, tampered cookies, invite-token leaks, self-heal                                                                                                                                                                                                            |
| Complaint suite (19)        | Who may report, duplicates, admin-only resolution, vote voiding & re-scoring, reputation updates, no email leaks                                                                                                                                                                                                                                                |
| Penalty suite (21)          | Removal blocks proof/review, false-report penalty, suspension below 25 on every write path, 4-day recovery, review credit, revoke after start, no removal after settlement                                                                                                                                                                                      |

API suites run against the **production build** on a throwaway copy of the database, so real data is never touched. `npm run check:env` verifies configuration (contract reachable, attester key matches `protocolAttester`) before every deploy.

| TARGET    | HOW                                                                                                                                                                                                                                                  |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Vercel    | <code>vercel.json</code> runs <code>scripts/vercel-build.js</code> : switch Prisma to PostgreSQL → generate → <code>db push</code> → <code>next build</code> . Uses Supabase Postgres (pooled <code>DATABASE_URL</code> + <code>DIRECT_URL</code> ). |
| Docker    | Multi-stage image with Next.js <code>standalone</code> output; SQLite on a volume; schema applied on start; health check on <code>/api/health</code> .                                                                                               |
| Contracts | <code>npm run deploy:sepolia</code> writes ABI + address book; older contracts stay routable.                                                                                                                                                        |

Serverless-friendly choices: no background jobs (reputation recovery and chain sync are lazy), stateless HMAC sessions, per-instance caches that are safe to lose, short RPC timeouts, and all secrets in environment variables.

| DECISION                                        | WHY                                        | TRADE-OFF                                 |
|-------------------------------------------------|--------------------------------------------|-------------------------------------------|
| Off-chain verification + signed settlement      | Cheap, private, flexible rules             | Trust in the attesor key                  |
| Pull payments                                   | Safe against griefing and reentrancy       | One extra transaction per user            |
| Integer math, round down, dust to treasury      | Never insolvent, deterministic             | Tiny amounts go to treasury               |
| Fee taken from the stake                        | Joiners pay exactly the advertised amount  | Net stake slightly below the headline     |
| Contract as source of truth, DB as cache        | No drift can block settlement              | RPC reads (mitigated by cache)            |
| Redeploy instead of upgradeable proxy           | Simpler, no admin upgrade power over funds | Routing across versions                   |
| Penalties via attestation (0 periods)           | No contract change, works on all versions  | Relies on the attesor                     |
| Wallet sessions instead of signing every action | One free signature per week                | Cookie must be protected (httpOnly, HMAC) |

## Future work

- On-chain dispute window for attestations; threshold-signature attestors.
- Automatic proof checks (GitHub commits, Strava activities).
- Stake-weighted or randomly assigned reviewers to resist collusion.
- Gasless joins via ERC-4337 / meta-transactions; L2 deployment for lower fees.
- Notifications (email / push) for deadlines and reviews.

**Q Why not put proofs and votes on-chain?**

Gas for every action, public exposure of personal links, and inflexible rules. Only the outcome (periods per wallet) needs to be trust-minimised, so it is summarised into one signed message and verified on-chain.

---

**Q What stops the backend from stealing funds?**

It never holds funds or keys to the contract's balance. The contract only accepts a settlement signed by the attester, and even then it computes every payout itself and only lets each wallet withdraw its own balance.

---

**Q How is replay prevented?**

EIP-712 domain separation (chainId + contract address), a per-challenge settlement nonce incremented on finalize, and the finalized flag. Sign-in messages are single-use with a 10-minute TTL.

---

**Q Why sort addresses?**

A canonical order makes the signed message deterministic and lets the contract detect duplicates with one comparison ( $p > \text{last}$ ) instead of a nested loop.

---

**Q Explain the payout formula.**

Net stake  $S = \text{stake} - 0.125\%$ . Retained  $= S \cdot c/T$ . Missed parts form a pool split among qualifiers ( $c \cdot 100 \geq \text{threshold} \cdot T$ ) in proportion to  $c$ . If nobody qualifies the pool goes to the treasury.

---

**Q How do you guarantee solvency?**

All divisions floor; the difference between the pool and distributed rewards (dust) goes to the treasury; tests assert  $\Sigma \text{claimable} + \text{treasuryCut} + \text{fees} = n \cdot \text{stake}$  exactly.

---

**Q Why pull instead of push payments?**

Pushing in a loop lets one reverting recipient block everyone and adds reentrancy surface; pulling isolates each withdrawal.

---

**Q What is the zero-qualifier rule for?**

Without it, everyone could agree to skip and still split the pool. Sending the pool to the treasury removes that incentive.

---

**Q How does authentication work without passwords?**

The wallet signs a one-time message (EIP-191); the server verifies the signature, then issues an HMAC-signed httpOnly cookie for 7 days. Every write checks the session wallet.

---

**Q How do you keep the database and chain consistent?**

The contract is the source of truth: status and nonce are synced on read, joins and challenges are verified on-chain before being recorded, settlement reads the participant list from the contract, and missing challenges self-heal.

---

---

**Q What happens if the RPC is slow?**

Calls time out after 4 seconds; status falls back to a time-derived value; results are cached for 15 seconds; settled challenges never call RPC.

---

**Q How do penalties take someone's money without a contract change?**

Removed participants are attested with 0 completed periods, so the contract pays them nothing and their net stake goes to the pool. Removal is blocked after settlement.

---

**Q How does reputation recovery work without a cron job?**

Lazily: whenever a reputation is read, the server applies the +2 ticks owed for each full 4 days since lowSince and logs them.

---

**Q How did you make the frontend fast?**

Kept ethers out of the shared bundle (raw EIP-1193 + BigInt formatting), moved the ABI out of shared code, lazy-loaded Supabase, avoided per-request auth round-trips, cached chain reads and added indexes. Shared pages went from ~440 KB to ~112 KB.

---

**Q How do you handle contract upgrades?**

Deploy a new immutable version, start its IDs after the previous one's, and route each challenge ID to the contract that owns it via an address book.

---

**Q What are the biggest risks?**

Attestor key compromise and reviewer collusion/Sybil. Mitigations: key management and multisig, dispute windows, reputation, private cohorts, proof-of-personhood.

---

**Q Why Sepolia?**

It is the recommended Ethereum testnet: free test ETH, real EVM semantics, Etherscan support — safe for real users to try without risk.

---

**Q What would you change for mainnet?**

Audit, multisig owner and attestor, dispute window, L2 deployment, rate limiting, monitoring/alerts, and a formal privacy policy.

---

|                                                                              |                                                       |
|------------------------------------------------------------------------------|-------------------------------------------------------|
| contracts/CommitXProtocol.sol                                                | the protocol (v3)                                     |
| test/CommitXProtocol.test.js                                                 | 41 Hardhat tests                                      |
| scripts/deploy.js                                                            | deploy + write ABI and address book (auto ID offset)  |
| scripts/check-env.js                                                         | pre-deploy configuration check                        |
| scripts/vercel-build.js · use-postgres.js · db-transfer.js                   | Vercel / Postgres tooling                             |
| prisma/schema.prisma                                                         | data model                                            |
| src/config/contracts.js · deployments.js                                     | ABI · address book                                    |
| src/lib/ chain · chainSync · attestation · attestor · session · auth · api   |                                                       |
| reputation · moderation · proofStatus · challengeRecord · network · identity |                                                       |
| src/app/api/...                                                              | route handlers (see Backend)                          |
| src/app/...                                                                  | pages · src/components/... UI · src/context/... state |
| docs/DEPLOYMENT.md                                                           | operations guide                                      |